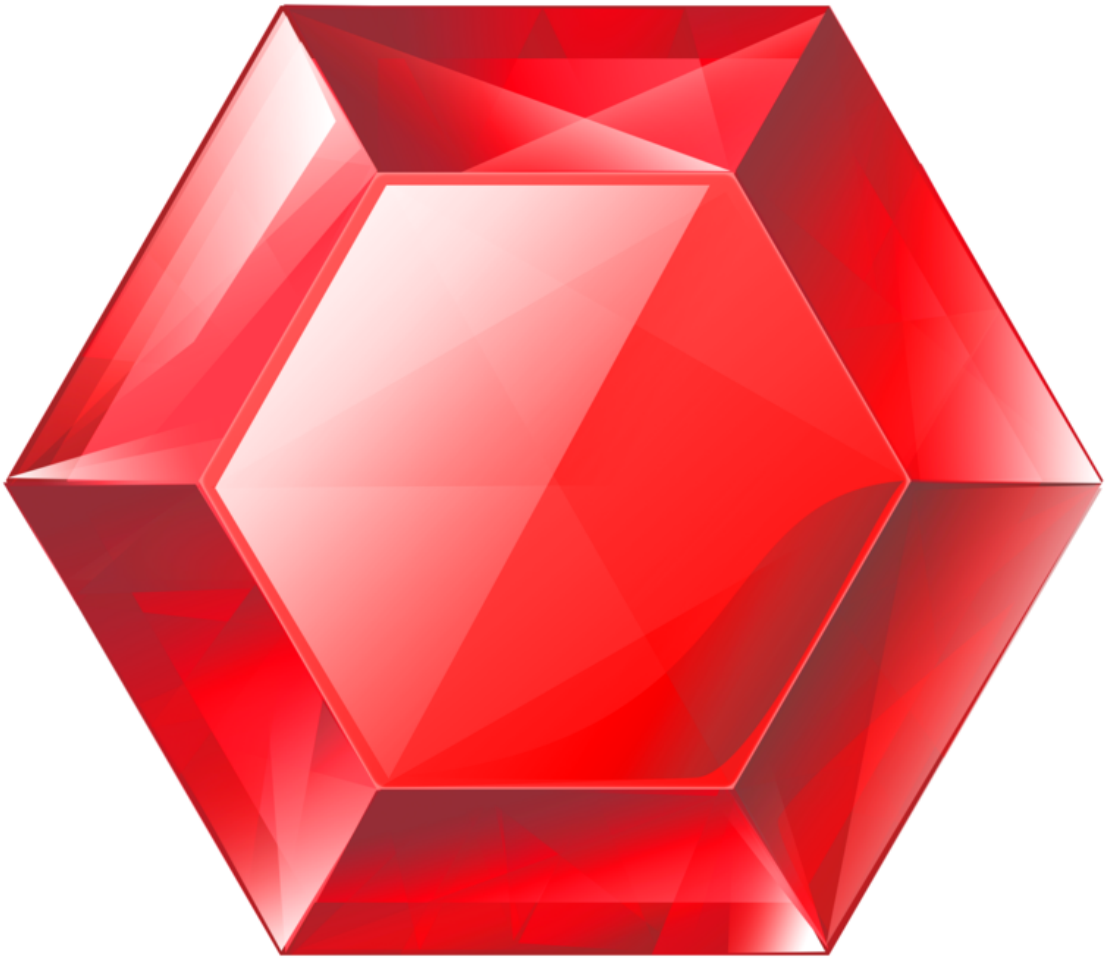
gemstone-rs Explorer Guide

Local HTTP explorer endpoints and safety defaults



Generated from the Markdown sources in gemstone-rs/docs.

Contents

Explorer Guide

`gemstone-rs-explorer` is a local-only HTTP explorer built on the Rust API. It is useful for proving the browser, inspect, eval, and codegen surfaces before embedding richer tooling in VS Code or another frontend.

Install and Run

```
cargo install gemstone-rs-explorer
gemstone-rs-explorer --port 8787
gemstone-rs-explorer --env-file .env.gemstone-rs --port 8787
```

From a checkout:

```
cargo run -p gemstone-rs-explorer -- --port 8787
```

If you launch the explorer outside the project checkout, point codegen discovery and relative config paths at the checkout explicitly:

```
gemstone-rs-explorer --port 8787 --codegen-root /path/to/gemstone-rs
```

Open:

```
http://127.0.0.1:8787/
```

The home page is a small browser UI over the same JSON endpoints. It can:

- browse dictionaries, classes, protocols, methods, and source
- show the current browse path, class-side toggle, result count, and selected

  method source in the detail pane

- run a setup assistant that checks the env file, live configuration, GCI

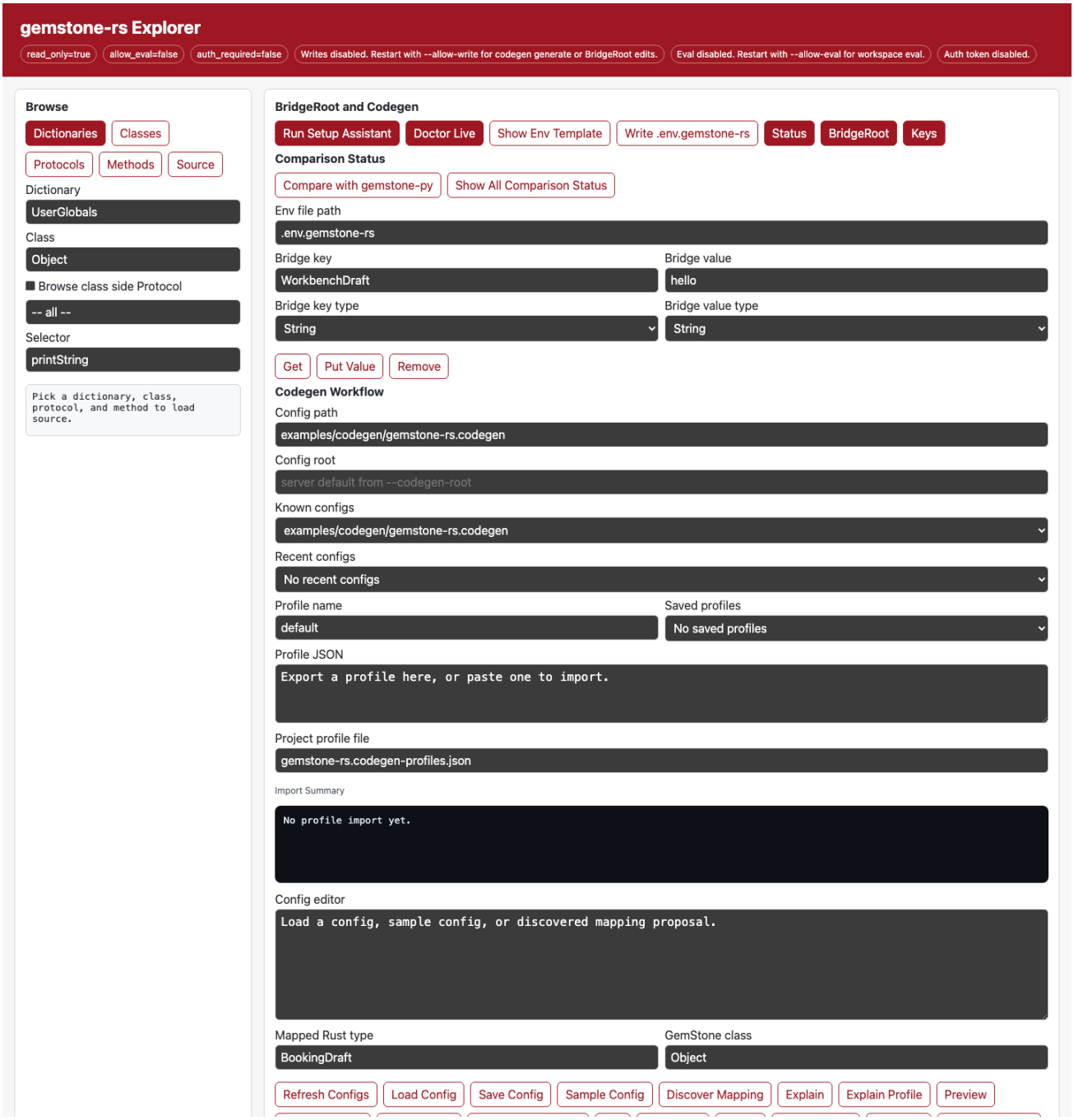
  library, codegen config, and project profiles

- run doctor/status checks
- compare gemstone-rs with gemstone-py and show the remaining batch/hour

  estimate without opening the terminal

- inspect BridgeRoot, list keys, and render value payload summaries
- run codegen sample/discover/preview/diff/check/generate from an editable config path
- refresh a picker of known `.codegen` files and load one without typing the path by hand
- set a project codegen root through `--codegen-root`, or override it from the page before refreshing configs
- reuse recently loaded, saved, previewed, diffed, checked, or generated config paths from local browser storage
- save named codegen profiles containing config root, config path, mapped Rust type, and GemStone class
- export and import codegen profiles as versioned JSON for sharing between browsers or teammates
- load project profile files from the codegen root, and save them back when the explorer runs with `--allow-write`
- load and save the selected codegen config file; saving is still gated by `--allow-write`, accepts a POST body for larger configs, and validates the config before writing
- render generated wrapper source, generated mapping config, colored unified diff output, current generated output files, and a side-by-side diff in a dedicated detail pane
- remember the current browser fields locally across reloads
- put/remove BridgeRoot values with explicit string/symbol key policy and string/small-int/bool value policy when `--allow-write` is enabled

Screenshot:



Refresh it with:

```
make screenshots
```

See Screenshot Workflow for the repeatable capture process.

Safety Defaults

The explorer:

- binds to `127.0.0.1` by default
- rejects non-loopback hosts
- starts read-only
- requires `--allow-eval` before workspace evaluation
- requires `--allow-write` before codegen writes
- supports an optional local auth token with `--auth-token` or `--auth-token-env`
- reads GemStone credentials from the same `GS_*` environment as the CLI

Do not expose it publicly. The token option is for local defense in depth on a loopback machine; it is not a substitute for TLS, user accounts, or a production auth layer.

To require a token for all explorer pages and APIs except `/health`:

```
export GEMSTONE_RS_EXPLORER_TOKEN='replace-with-a-local-random-token'
gemstone-rs-explorer --auth-token-env GEMSTONE_RS_EXPLORER_TOKEN
```

Then use either a query token for browser workflows:

```
open 'http://127.0.0.1:8787/?token=replace-with-a-local-random-token'
curl -s 'http://127.0.0.1:8787/api/config?token=replace-with-a-local-random-token'
```

Or a header for scripts:

```
curl -s \
  -H 'X-GemStone-RS-Token: replace-with-a-local-random-token' \
  http://127.0.0.1:8787/api/config
curl -s \
  -H 'Authorization: Bearer replace-with-a-local-random-token' \
  http://127.0.0.1:8787/api/status
```

Browse Endpoints

Run these from a second terminal:

```
curl -s http://127.0.0.1:8787/health
curl -s http://127.0.0.1:8787/api/config
curl -s 'http://127.0.0.1:8787/api/setup/assistant?config=examples/codegen/gemstone-
rs.codegen&profile_file=examples/codegen/gemstone-rs.codegen-profiles.json'
```

```
curl -s http://127.0.0.1:8787/api/doctor
curl -s 'http://127.0.0.1:8787/api/doctor?live=1'
curl -s http://127.0.0.1:8787/api/status
curl -s http://127.0.0.1:8787/api/browse/dictionaries
curl -s 'http://127.0.0.1:8787/api/browse/classes?dictionary=UserGlobals'
curl -s 'http://127.0.0.1:8787/api/browse/protocols?class=Object&meta=0'
curl -s 'http://127.0.0.1:8787/api/browse/methods?class=Object&protocol=--%20all%20--
&meta=0'
curl -s 'http://127.0.0.1:8787/api/browse/source?
class=Object&selector=printString&meta=0'
curl -s 'http://127.0.0.1:8787/api/inspect?oop=20'
```

Expected shapes:

```
{"status":"ok"}
{"host":"127.0.0.1","port":8787,"readOnly":true,"allowEval":false}
{"success":true,"environment":{"GS_PASSWORD":{"status":"set","masked":true}}}
{"connected":true,"sessionId":12345,"needsCommit":false,"inTransaction":false}
{"success":true,"dictionaries":["UserGlobals"]}
```

Eval

Eval is disabled by default:

```
gemstone-rs-explorer --allow-eval
```

Then:

```
curl -s 'http://127.0.0.1:8787/api/eval?source=3%20%2B%204'
```

Expected:

```
{"type":"smallInt","value":7}
```

Codegen Endpoints

The setup actions on the home page can show the same safe `GS_*` environment template as `gemstone-rs env sample`, or write `.env.gemstone-rs` when the explorer was started with `--allow-write`. Start the explorer with `--env-file .env.gemstone-rs` to use that file for all live browse, inspect, BridgeRoot, and codegen discovery actions. `Run Setup Assistant` checks the env file, GemStone configuration, GCI library, selected codegen config, and project profile file in one response.

The home page includes a Codegen Workflow panel. Set `Config path`, optionally set `Config root`, or start the server with `--codegen-root`. `Refresh Configs` and the `Known configs` picker choose an existing `.codegen` file relative to that root. The `Recent configs` picker remembers paths used by Load, Save, Preview, Diff, Check, and Generate in local browser storage. `Saved profiles` store the root, config path, mapped Rust type, and GemStone class as a named local browser profile. `Export Profile` writes a versioned JSON payload to `Profile JSON`; paste that JSON into another explorer and use `Import Profile` to merge it into saved profiles. `Load Project Profiles` reads `gemstone-rs.codegen-profiles.json` from the codegen root by default, while `Save Project Profiles` writes the current saved profiles back to that file when the explorer runs with `--allow-write`. Optionally enter a mapped Rust type and GemStone class, then use:

- `Config root` to override the server default for config discovery and

  relative config paths

- `Refresh Configs` to scan the project root for known `.codegen` files
- `Recent configs` to return to the last few config paths used in this browser
- `Save Profile` and `Saved profiles` to switch between named codegen workflows
- `Export Profile` and `Import Profile` to share a codegen workflow as JSON
- `Load Project Profiles` and `Save Project Profiles` to use a committed

  project-level profile file; the server rejects invalid profile schemas and the browser reports which profiles are new, replaced, or unchanged

- `Check Project Profiles` to verify every profile in the project profile file

  and show ok/stale/error counts in one report; the detail pane renders a table with profile name, config path, output file, freshness, and per-profile Preview, Diff, Check, and Generate actions

- `Load Config` to read the selected config file into the editor
- `Save Config` to POST the editor contents, validate them, and write the file

  when the explorer was started with `--allow-write`

- `Sample Config` to view the starter config format
- `Discover Mapping` to ask a live stone for a mapping config proposal
- `Explain` to render a structured codegen summary, including output path,

  generated test stubs, wrappers, return helpers, and mapped fields

- `Explain Profile` to render the same structured summary after resolving a

  named profile from the project profile file

- `Preview` to inspect generated Rust wrappers without writing files
- `Preview Profile` to inspect generated wrappers after resolving a named

  project profile

- `Read Output` to load the currently committed generated output file named by the config, without regenerating it
- `Read Profile Output` to load the generated output file after resolving a named project profile
- `Diff` to compare generated output with the committed file; the detail pane shows both the exact unified diff and a side-by-side view for review
- `Diff Profile` to compare generated output from a named project profile
- `Check` to fail when generated output is stale
- `Check Profile` to run the stale-output check from a named project profile
- `Generate` to write wrappers when the explorer was started with `--allow-write`
- `Generate Profile` to resolve a named project profile and write its wrappers when the explorer was started with `--allow-write`

The VS Code webview wraps the same page with native handoff actions. It can run live browse probes from the side inspector, open method source in a VS Code editor, open the configured codegen/profile files, open the generated output file reported by codegen explain/check/profile status, render generated wrapper source in an editable webview textarea, open that text as an untitled VS Code draft, and save edited generated output back to the configured output path after a confirmation prompt. The save path is constrained to the configured checkout or workspace. The webview also offers Launch Explorer/Open Browser/Copy URL fallbacks when the loopback explorer is not running.

The `Comparison Status` buttons call read-only local endpoints:

- `Compare with gemstone-py` renders the same short answer as `gemstone-rs compare gemstone-py --status`
- `Show All Comparison Status` renders the active gemstone-rs batch count, currently **0 batches** and roughly **0-0 hours**

Read-only endpoints:

```
curl -s http://127.0.0.1:8787/api/compare/gemstone-py/status
curl -s http://127.0.0.1:8787/api/compare/all/status
curl -s http://127.0.0.1:8787/api/codegen/sample
curl -s 'http://127.0.0.1:8787/api/codegen/configs?root=.'
curl -s 'http://127.0.0.1:8787/api/codegen/profiles?profile_file=gemstone-rs.codegen-
profiles.json'
curl -s 'http://127.0.0.1:8787/api/codegen/profiles/check?profile_file=examples/
```

```
codegen/gemstone-rs.codegen-profiles.json'
curl -s 'http://127.0.0.1:8787/api/codegen/config?config=examples/codegen/gemstone-
rs.codegen'
curl -s 'http://127.0.0.1:8787/api/codegen/explain?config=examples/codegen/gemstone-
rs.codegen'
curl -s 'http://127.0.0.1:8787/api/codegen/explain-profile?
profile=default&profile_file=examples/codegen/gemstone-rs.codegen-profiles.json'
curl -s 'http://127.0.0.1:8787/api/codegen/preview?config=examples/codegen/gemstone-
rs.codegen'
curl -s 'http://127.0.0.1:8787/api/codegen/preview-profile?
profile=default&profile_file=examples/codegen/gemstone-rs.codegen-profiles.json'
curl -s 'http://127.0.0.1:8787/api/codegen/diff?config=examples/codegen/gemstone-
rs.codegen'
curl -s 'http://127.0.0.1:8787/api/codegen/diff-profile?
profile=default&profile_file=examples/codegen/gemstone-rs.codegen-profiles.json'
curl -s 'http://127.0.0.1:8787/api/codegen/output?config=examples/codegen/gemstone-
rs.codegen'
curl -s 'http://127.0.0.1:8787/api/codegen/output-profile?
profile=default&profile_file=examples/codegen/gemstone-rs.codegen-profiles.json'
curl -s 'http://127.0.0.1:8787/api/codegen/check?config=examples/codegen/gemstone-
rs.codegen'
curl -s 'http://127.0.0.1:8787/api/codegen/check-profile?
profile=default&profile_file=examples/codegen/gemstone-rs.codegen-profiles.json'
curl -s 'http://127.0.0.1:8787/api/codegen/discover-mapping?
mapped=BookingDraft&class=Object'
```

The repository includes a sample project profile file:

```
examples/codegen/gemstone-rs.codegen-profiles.json
```

Expected read-only results include the config list and a current generated output check:

```
{"success":true,"root":".","configs":["examples/codegen/gemstone-rs.codegen"]}
{"success":true,"exists":true,"upToDate":true}
{"success":true,"ok":true,"profileCount":3,"okCount":3,"staleCount":0,"errorCount":0}
```

Write-gated endpoint:

```
gemstone-rs-explorer --allow-write
```

```
curl -s 'http://127.0.0.1:8787/api/codegen/generate?config=examples/codegen/gemstone-
rs.codegen'
curl -s 'http://127.0.0.1:8787/api/codegen/generate-profile?
profile=default&profile_file=examples/codegen/gemstone-rs.codegen-profiles.json'
curl -s -X POST \
  'http://127.0.0.1:8787/api/env/write?path=.env.gemstone-rs'
curl -s -X POST \
  --data-binary @gemstone-rs.codegen-profiles.json \
  'http://127.0.0.1:8787/api/codegen/profiles/save?profile_file=gemstone-rs.codegen-
```

```
profiles.json'
curl -s -X POST \
  --data-binary @examples/codegen/gemstone-rs.codegen \
  'http://127.0.0.1:8787/api/codegen/config/save?config=examples/codegen/
draft.codegen'
```

Expected:

```
{"success":true,"output":"examples/codegen/generated/
gemstone_wrappers.rs","bytes":1234}
{"success":true,"config":"examples/codegen/draft.codegen","bytes":512}
```

Project profile files use this shape:

```
{"kind":"gemstone-rs-explorer-codegen-profiles","version":1,"profiles":
[{"name":"default","config":"examples/codegen/gemstone-
rs.codegen","root":"","mapped":"BookingDraft","className":"Object"}]}
```

Validate the file before sharing or saving it from CI:

```
gemstone-rs profile sample
gemstone-rs profile init gemstone-rs.codegen-profiles.json
gemstone-rs profile validate gemstone-rs.codegen-profiles.json
gemstone-rs profile validate --json gemstone-rs.codegen-profiles.json
gemstone-rs profile list gemstone-rs.codegen-profiles.json
gemstone-rs profile show default gemstone-rs.codegen-profiles.json
gemstone-rs profile resolve default gemstone-rs.codegen-profiles.json
gemstone-rs profile check gemstone-rs.codegen-profiles.json
gemstone-rs profile check --json gemstone-rs.codegen-profiles.json
gemstone-rs codegen check-profile default gemstone-rs.codegen-profiles.json
```

The schema reference is Codegen Profile Schema.

For write endpoints, relative `config=` and `profile_file=` paths are resolved inside the codegen root. Paths containing `..` are rejected after URL decoding, `root=...` traversal is rejected as well, and absolute write paths require starting the explorer with `--allow-absolute-write-paths`.

The browser UI also asks for confirmation before BridgeRoot writes, env-file writes, profile saves, config saves, and codegen generate actions. Server-side guards still decide the real access policy: write endpoints return `403` unless the explorer was started with `--allow-write`.

Project profile saves validate the JSON before writing. The top-level object must contain only `kind`, `version`, and `profiles`; profile names are required and unique; and each profile may contain only string-valued `name`, `config`, `root`, `mapped`, and `className` fields.

BridgeRoot Endpoints

BridgeRoot inspection and mapping-config preview use the same live session configuration as the browser endpoints:

```
curl -s http://127.0.0.1:8787/api/bridge/root
curl -s http://127.0.0.1:8787/api/bridge/keys
curl -s 'http://127.0.0.1:8787/api/bridge/get?key=BookingDraft'
curl -s 'http://127.0.0.1:8787/api/bridge/get?key=BookingDraft&depth=4'
curl -s 'http://127.0.0.1:8787/api/bridge/get?key=BookingDraft&key_type=Symbol'
curl -s 'http://127.0.0.1:8787/api/bridge/shape?key=BookingDraft&depth=4'
curl -s 'http://127.0.0.1:8787/api/bridge/mapping-config?mapped=BookingDraft'
curl -s 'http://127.0.0.1:8787/api/bridge/mapping-preview?
key=BookingDraft&mapped=BookingDraft&depth=4'
```

Use these endpoints as a deliberate mapping workflow:

1. Inspect the live value with `/api/bridge/get`.
2. Review the relationship paths and repeated identities with

   `/api/bridge/shape`.

3. Create a first mapping config with `/api/bridge/mapping-preview`.
4. Review the generated config before writing files or committing codegen

   output.

The browser UI exposes `Bridge key type` and `Bridge value type` controls so you can explicitly test string-key, symbol-key, string-value, symbol-value, small-int-value, and bool-value mappings before encoding them in a reusable config file. `BridgeRoot Value` now shows both the raw OOP/class/printString summary and a nested `BridgeValue` tree for string/symbol-keyed dictionaries and arrays. `Preview Mapping Config` reads the same nested value and writes a starter `mapped = ...` / `field = ...` config into the detail pane, including review notes for opaque OOPs, nil fields, symbols, empty arrays, and mixed arrays. `Shape Report` turns the value into a relationship table with paths, node kinds, key policy, child counts, nil counts, opaque OOP counts, report-local identity ids, and repeated opaque references. It also groups repeated opaque OOPs so the UI can show every path pointing at the same GemStone object. The page persists the selected key, value, class, selector, config, and mapping fields in browser local storage; use `Clear Saved Fields` to reset the page back to the documented defaults.

The explorer does not turn GemStone objects into transparent local state. Object identity stays visible as OOPs, unsupported values remain opaque in the `BridgeValue` tree, and writes remain behind the `--allow-write` gate. Rust-side `Remote<T>` handles use the same explicit model: inspect first, materialize with a profile, then call `refresh` or `save` with `&mut Session`.

The same nested read-back is available from the CLI:

```
gemstone-rs bridge value BookingDraft --depth 4
gemstone-rs bridge shape BookingDraft --depth 4
gemstone-rs bridge mapping-preview BookingDraft --mapped BookingDraft --depth 4
```

Writes are disabled unless the explorer is started with `--allow-write`:

```
gemstone-rs-explorer --allow-write
curl -s 'http://127.0.0.1:8787/api/bridge/put?key=WorkbenchDraft&value=hello'
curl -s 'http://127.0.0.1:8787/api/bridge/put?
key=WorkbenchState&value=ready&value_type=Symbol'
curl -s 'http://127.0.0.1:8787/api/bridge/put?
key=WorkbenchCount&value=7&value_type=SmallInt'
curl -s 'http://127.0.0.1:8787/api/bridge/put?
key=WorkbenchApproved&value=true&value_type=Bool'
curl -s 'http://127.0.0.1:8787/api/bridge/remove?key=WorkbenchDraft'
```

Expected shapes:


```
{"success":true,"name":"GemStoneRsBridgeRoot","oop":1234,"identityId":1}
{"success":true,"root":"GemStoneRsBridgeRoot","keys":
[{"printString":"BookingDraft"}]}
{"success":true,"root":"GemStoneRsBridgeRoot","key":"BookingDraft","keyType":"String"
,"depth":4,"value":
{"oop":5678,"classOop":9012,"printString":"aDictionary(...)"},"bridgeValue":
{"type":"dictionary","entries":{"name":{"type":"string","value":"Tariq"}}}}
{"success":true,"root":"GemStoneRsBridgeRoot","key":"BookingDraft","shape":
{"totalNodes":6,"nodes":
[{"path":"value.customer.#name","kind":"string"}],"identityGroups":
[{"identityId":1,"oop":1234,"paths":["value.items[2]","value.items[3]"]}]}}
{"success":true,"config":"mapped = BookingDraft ..."}
{"success":true,"root":"GemStoneRsBridgeRoot","key":"BookingDraft","mapped":"BookingD
raft","config":"# Generated by gemstone-rs from a live BridgeValue ..."}
{"success":true,"root":"GemStoneRsBridgeRoot","key":"WorkbenchDraft","keyType":"Strin
g","valueType":"String","oop":3456}
```


Product Direction

The explorer should remain a separate binary crate. The core `gemstone-rs` library stays focused on safe GemStone API access, while the explorer proves higher-level browsing and codegen workflows.

Good next steps:

- keep Marketplace/GitHub screenshots current after UI changes
- run the explorer endpoint smoke checks and VS Code smoke checks after

  touching the embedded webview

- refine workflow feedback around writes, generated-file saves, and live

  BridgeRoot probes as real projects expose rough edges

> This PDF was generated locally from `docs/`. If you edit the Markdown sources, rerun `python docs/build_pdf_docs.py`.